

Final Report

By: Johnny Konagel(jmkonag) and Winston Ceh(ArloTGG)

Introduction

In our project, we decided to demonstrate how a honeyfile can be used as “proactive” security and a digital forensics tool. A honeyfile is a decoy file that is placed within a dead system to attract attackers. It is specifically designed to get “clicks” by attackers, which then relays information to us in order to prevent this from happening in the future. In digital forensics specifically, one of the biggest challenges is detecting breaches early enough to minimize the loss of sensitive data. General security implements measures in order to prevent breaches completely, but doesn’t have immediate control of the situation. In this case, a Honeyfile is used as a proactive tool, ensuring that future issues can be avoided completely, while also being prepared in the case of a breach. By using honeyfiles, investigators and security teams have the opportunity to:

- Gather evidence for analysis
- Identify insider threats and external forces
- Detect unauthorized access quickly and promptly
- Improve overall incident response time

Technical Implementation

Our project is primarily written in python being run from two different files, “deployment.py” and “log_hash.py.” We also required the use of two other python libraries, cryptography for our hashing and encryption of the audit logs generated and google-genai to allow for Gemini implementation. As said, we chose to use Gemini 3 Flash Preview as the AI model to analyze and list the contents of the audit log as it had the best response time and low error rates compared to other models we have tested. We also chose python as our language of choice as it has easy implementation and automation into Linux file systems as well as allowing us to perform the tasks needed with our honeyfile system.

Our honeyfile system itself is primarily using “auditd” which allows you to create the honeyfile trap and designate what file is the one you should be checking. In our case, we created “pass.txt” and placed it into the system’s /etc folder to mimic a fake password file a malicious actor might try and read. Any activity involving this file, such as permission changes, are stored in an audit log in /var/log/audit. This log is then passed through every day to our “log_hash.py” file using crontab which encrypts and hashes the file, storing it in a nearby folder with timestamps to allow for previous callbacks. The file also is able to use an already premade key set or create its own if one does not exist for the hashing and encryption if needed.

For our testing purposes, we used both the main admin, “vmuser,” to try and modify the pass.txt as well as another created user by the name of “eri” who would also try and modify the file.

Results

After some time, we were successful in getting the honeyfile to correctly log actions taken on it and pass it through our hashing and deployment python file.

```

### Summary of Activities

| Timestamp | User ('audit'/'uid') | Process Name | Action Taken | Success | Result/Outcome |
| :--- | :--- | :--- | :--- | :--- | :--- |
| 14:59:59 | 'vmuser' / 'root' | 'chmod' | 'chmod 777 /etc/pass.txt' | **Yes** | Permission change successful. |
| 15:00:08 | 'vmuser' / 'eri' | 'chmod' | 'chmod 777 /etc/pass.txt' | **No** | Denied (EPERM - Operation not permitted). |

---

### Detailed Analysis

#### Incident 1: Successful Modification by Root
* **User Context:** The 'audit' (Audit User ID) is 'vmuser', but the effective 'uid' is 'root'. This indicates that the user 'vmuser' escalated privileges to 'root' (likely via 'sudo') before executing the command.
* **Action:** The user executed 'chmod 777' on the sensitive file '/etc/pass.txt'.
* **Context:** The current working directory was '/home/vmuser'. The system successfully applied the wide-open permissions (777) to the file, which is a classic indicator of an attacker attempting to prepare a file for unauthorized read/write access.

#### Incident 2: Unauthorized Attempt by 'eri'
* **User Context:** The 'audit' is 'vmuser', and the effective 'uid' is 'eri'. This suggests the same logged-in session ('ses=3') attempted to perform an action as a different local user.
* **Action:** The user attempted to execute 'chmod 777' on the same file ('/etc/pass.txt').
* **Context:** The current working directory was '/home/eri'. The syscall returned 'EPERM' (Operation not permitted).
* **Analysis:** This demonstrates that the file system permissions on '/etc/pass.txt' successfully prevented user 'eri' from modifying the file, triggering the 'auditd' trap.

### Security Observations
1. **Honeytrap Efficacy:** The 'honeyfile_trap' is working as intended, successfully logging both successful and blocked attempts to touch the monitored file.
2. **Session Correlation:** Both events share 'ses=3', confirming that the same initial login session ('vmuser') is responsible for both the root escalation and the subsequent attempt by the 'eri' account.
3. **Pattern:** The attempt to change the file to '777' is a strong indicator of malicious intent, specifically an attempt to bypass file access controls to either exfiltrate data or inject malicious content into the target file.

vmuser@vmuser-Standard-PC-i440FX-PIIX-1996:~/Downloads$ sudo python3 log_hash.py
[sudo] password for vmuser:
[+] Success!
- Encrypted log: /var/secure_logs/encrypted/audit_log_20260502_150233.enc
- SHA-256 Hash: a7fcbe33a340ee8664eaf41d3f254b68210e84e0477498a52f655f5b936b
n2f8d

```

The first two images showcase the output of the “deployment.py” file which shows the analysis of our audit log produced by the honeyfile system. We can see it was able to read our taken actions from both users used in testing. Even in cases of it being unsuccessful, it still logs it as shown with eri. The observations are included at the end suggesting what might be the case or severity of the actions captured in the audit log, even being able to suggest privilege escalation due to the same session being detected.

Lastly, we see the output of the “log_hash.py” file which was able to successfully encrypt and hash the file, storing it in “/var/secure_logs/encrypted/” with the time stamp. This was manually run to showcase the results but the file is running every day in crontab and showcased in our demo video located on github.

However, this was not without difficulty as we had different plans initially in this project. At the start, we wanted the AI analysis to differentiate between accidental and malicious users in the system caught by our honeyfile trap. In our findings this was not the case and only reported back everything as potentially malicious, suggesting root escalation or horizontal movement. Additionally, the audit log had to be

shortened to the most recent 200 lines stored in the log otherwise the input would be much too large for Gemini to read and produce errors.

Conclusion

Throughout the implementation of our project, we ran into a couple of issues. The first main issue we had was the implementation of the API key. First, we began with errors ensuring a secure and consistent key. The API key seemed to have issues with consistent and reliable feedback, so we shifted our focus to “Gemini-3.1-flash-lite-preview”, which was a good option for us. Another issue we had was the minimal scope of testing. Our honeyfile was placed in a controlled environment, therefore being limited to our possible scope of activity.

One thing that we learned was the importance of implementation details, especially when working with things such as API keys. Setting up the API correctly, managing permissions, and securely storing keys were more challenging than expected. This showed us that even the smallest configuration issues can have a big impact on whether a system even works in the long run. Another key takeaway was how important it is to have security tools like this within a database. Honeyfiles help for both preparation and response in an attack scenario. Therefore, the use of this technique is helpful in multiple different scenarios. Finally, we also learned the value of testing in different scenarios. We worked within a controlled environment, so that helped us understand the basics. Now, we’d like to expand on this and see how it would operate on a much larger database, one of which has real-world implications.

***We both worked on deployment.py and loghash.py; The history in the “commits” section is disproportionate because Winston was refining the README file and the organization of the GitHub.**